

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1105

COURSE OUTCOME COVERED

CO5: Analyze object-oriented software using quality assurance and usability testing Techniques (BL4, BL5)
Assessment Tool Weightage: 25%

QUESTIONS: 05

Total Marks:25

Annexure A

Error Analysis Task

Code of Conduct:

I RASHMI NAURENE (192521357) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

RUBRICS FOR CALCULATION:

Error Analysis Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q4. Scenario: Smart Parking Management System

Modules:

User Registration, Parking Slot Search, Slot Reservation, Payment, Vehicle Entry/Exit

Question:

Develop **pseudocode for constraint-based error analysis** in the parking reservation process. The booking must satisfy conditions such as valid vehicle details, available parking slots, valid reservation duration, and successful payment. Demonstrate how invalid conditions are detected, handled, and logged.

Q5. Scenario: Online Examination System

Features:

Student Login, Exam Selection, Question Display, Answer Submission, Result Processing

Question:

Write **pseudocode for integrating quality assurance and usability testing** in the online examination system. The process should verify functional correctness of login, question delivery, answer submission,

and result processing while also evaluating response time, navigation difficulty, accessibility, and user errors. Explain how the collected results can be analyzed to identify system defects and usability issues.

Q4

Smart Parking Management System - Constraint-Based Error Analysis

Error Analysis Constraints:

Valid Vehicle Details: Registration number exists and active. Available Parking Slots: Requested type has free slots. Valid Duration: 1-24 hours, no time conflicts. Successful Payment: Gateway confirmed, amount correct, transaction ID generated.

Pseudocode - Constraint Validation:

```
FUNCTION validateParkingReservation(vehicleReg, slotType, duration, amount)
    errors = EMPTY_LIST

    // Validate Vehicle Details
    IF vehicleReg IS NULL
        errors.ADD("E001: Vehicle registration missing")
    ELSE IF NOT validateFormat(vehicleReg)
        errors.ADD("E002: Invalid vehicle format")
    ELSE IF NOT vehicleExists(vehicleReg)
        errors.ADD("E003: Vehicle not in system")
    ELSE IF NOT vehicleIsActive(vehicleReg)
        errors.ADD("E004: Vehicle inactive/expired")
    END IF

    // Validate Slot Availability
    IF slotType IS NULL
        errors.ADD("E005: Slot type not specified")
    ELSE IF NOT isValidType(slotType)
        errors.ADD("E006: Invalid slot type")
    ELSE IF countAvailableSlots(slotType) == 0
        errors.ADD("E007: No available slots")
    END IF

    // Validate Duration
    IF duration < 1 OR duration > 24
        errors.ADD("E008: Duration must be 1-24 hours")
    ELSE
        IF hasTimeConflict(slotId, startTime, endTime)
            errors.ADD("E009: Time conflict with existing booking")
        END IF
    END IF
```

```

        END IF
    END IF

    // Validate Payment
    expectedAmount = getHourlyRate(slotType) * duration
    IF amount != expectedAmount
        errors.ADD("E010: Payment amount incorrect")
    ELSE
        response = processPayment(amount)
        IF response.status != SUCCESS
            errors.ADD("E011: Payment failed")
        ELSE IF response.transactionID IS NULL
            errors.ADD("E012: No transaction ID")
        END IF
    END IF

    // Log Results
    IF errors.isEmpty()
        logSuccess("Reservation created")
        RETURN {status: SUCCESS, reservationID: id}
    ELSE
        logErrors(errors)
        RETURN {status: FAILED, errors: errors}
    END IF
END FUNCTION

```

Error Classification:

Critical (E001-E004, E010-E012): Block reservation. User action needed. Medium (E005-E007, E009): Constraint violated. Suggest alternative. Logging: All errors recorded with timestamp, user ID, error code for audit trail and system monitoring.

online shopping

RUN COMMANDS...CTRL + R

Explorer

Healthcare

Model

smart healthcare

Model.uml

Model

Use Case Diagram

<Package Import> PrimitiveTypes

customer

delivery system

third party payment

search products

manage order

manage account

manage shopping cart

payment

Usage

Usage

Association

Association

Usage

Send Feedback

Contact Us!

Use Case Diagram xActivity Diagram x

activity: Parking Reservation - Error Flow

ActivityParameterNode1

Submit Reservation Request

Valid Details, Slot Available, Valid Duration & Payment
else reservation

Yes

Confirm Reservation

No

Show Error Message & % Log Error

React Flow

Details

Activity Diagram

Core Properties

Label

Activity Diagram

Documentation

Documentation

Description NameActivity Diagram

SANDBOX VERSION - DATABASE CLEARED REGULARLY

SANDBOX STARTING GUIDE

31°C
Mostly clear

Search

ENG
IN

20:24
20-08-2025

Online Examination System - Integrated QA and Usability Testing

Testing Integration:

QA Tests (Functional): Login validation, question delivery accuracy, answer submission correctness, result calculation accuracy. Usability Tests: Response time <2s, navigation <3 steps, accessibility (WCAG AA), user error rate <5%. Goal: Identify both system defects and user experience problems simultaneously.

Pseudocode - Integrated QA & Usability:

```
FUNCTION integratedQAUsabilityTest()
    qaResults = EMPTY_LIST
    usabilityResults = EMPTY_LIST
    defects = EMPTY_LIST

    // PHASE 1: QA FUNCTIONAL TESTS

    FOR EACH loginTest IN testData.loginTests
        response = performLogin(studentID, password)
        IF response.sessionToken EXISTS
            qaResults.ADD({test: Login, status: PASS})
        ELSE
            qaResults.ADD({test: Login, status: FAIL})
            defects.ADD({code: QA001, type: CRITICAL, issue: Session not created})
        END IF
    END FOR

    FOR EACH exam IN testData.exams
        delivered = getExamQuestions(examID)
        expected = exam.questions
        IF delivered.count() == expected.count()
            qaResults.ADD({test: QuestionDelivery, status: PASS})
        ELSE
            qaResults.ADD({test: QuestionDelivery, status: FAIL})
            defects.ADD({code: QA002, type: CRITICAL, issue: Wrong question count})
        END IF
    END FOR

    FOR EACH submission IN testData.submissions
        startTime = NOW
        response = submitAnswer(examID, questionID, answer)
        submitTime = NOW - startTime
```

```

    saved = queryDB(SELECT answer FROM submissions)
    IF saved == answer
        qaResults.ADD({test: AnswerSubmission, status: PASS, time: submitTime})
    ELSE
        qaResults.ADD({test: AnswerSubmission, status: FAIL})
        defects.ADD({code: QA003, type: CRITICAL, issue: Answer not saved})
    END IF
END FOR

FOR EACH resultTest IN testData.results
    calculated = calculateScore(answers, key)
    expected = resultTest.expectedScore
    IF calculated == expected
        qaResults.ADD({test: ResultCalc, status: PASS, score: calculated})
    ELSE
        qaResults.ADD({test: ResultCalc, status: FAIL})
        defects.ADD({code: QA004, type: CRITICAL, issue: Score mismatch})
    END IF
END FOR

// PHASE 2: USABILITY TESTS

FOR EACH action IN testData.userActions
    startTime = NOW
    performAction(action)
    responseTime = NOW - startTime

    IF responseTime > 2000 // 2 seconds threshold
        usabilityResults.ADD({action: action, time: responseTime, status: SLOW})
        defects.ADD({code: USL001, type: MEDIUM, issue: Slow response})
    ELSE
        usabilityResults.ADD({action: action, time: responseTime, status: OK})
    END IF
END FOR

FOR EACH journey IN testData.navigationPaths
    steps = journey.stepCount
    IF steps > 3
        usabilityResults.ADD({path: journey.name, steps: steps, status:
COMPLEX})
        defects.ADD({code: USL002, type: LOW, issue: Too complex})
    ELSE
        usabilityResults.ADD({path: journey.name, steps: steps, status: OK})
    END IF
END FOR

```

```

FOR EACH element IN testData.screenElements
    contrast = calculateContrast(foreColor, bgColor)
    hasLabel = element.hasAriaLabel()
    keyboardOK = element.isKeyboardAccessible()

    IF contrast < 4.5 // WCAG AA standard
        usabilityResults.ADD({element: element.id, issue: Low contrast})
        defects.ADD({code: USL003, type: MEDIUM, issue: Accessibility:
contrast})
    END IF
    IF NOT hasLabel AND element.isInput
        usabilityResults.ADD({element: element.id, issue: No label})
        defects.ADD({code: USL004, type: MEDIUM, issue: Accessibility: missing
label})
    END IF
    IF NOT keyboardOK AND element.isInteractive
        usabilityResults.ADD({element: element.id, issue: Not keyboard
accessible})
        defects.ADD({code: USL005, type: MEDIUM, issue: Accessibility: no
keyboard nav})
    END IF
END FOR

FOR EACH session IN testData.userSessions
    userErrors = trackErrors(session)
    errorRate = (userErrors.count / session.actionCount) * 100

    IF errorRate > 5%
        usabilityResults.ADD({session: session.id, errorRate: errorRate, status:
HIGH})
        defects.ADD({code: USL006, type: LOW, issue: High user error rate})
    ELSE
        usabilityResults.ADD({session: session.id, errorRate: errorRate, status:
OK})
    END IF
END FOR

// PHASE 3: GENERATE REPORT
qaPass = countPassing(qaResults)
qaTotal = qaResults.count()
qaPassRate = (qaPass / qaTotal) * 100

criticalCount = countDefectsBy(defects, CRITICAL)
highCount = countDefectsBy(defects, HIGH)
mediumCount = countDefectsBy(defects, MEDIUM)
lowCount = countDefectsBy(defects, LOW)

```

```

report = {
    qaPassRate: qaPassRate + "%",
    defectSummary: {Critical: criticalCount, High: highCount, Medium:
mediumCount, Low: lowCount},
    avgResponseTime: calculateAverage(usabilityResults.time),
    accessibilityCompliance: calculateCompliancePercent() + "% (WCAG AA)",
    userErrorAvg: calculateAverageErrorRate() + "%"
}

RETURN {qa: qaResults, usability: usabilityResults, defects: defects,
report: report}

END FUNCTION

```

Key QA & Usability Metrics:

QA: Login success (target 100%), Question accuracy (100%), Answer save success (>99%), Result calc accuracy (100%). Usability: Avg response <2s, Navigation <3 steps, Accessibility compliance >95% (WCAG AA), User error rate <5%. Critical defects: Fix immediately. High: 1 sprint. Medium/Low: Future sprints.

